

Assessment 2	Backend implementation and database integration
Due date	11.59 pm, Sunday 13 September 2026
Weighting	25%
Word count/length	1,100 words equivalent
Generative AI use	Full AI
Individual	This is an individual assessment.
Feedback	Written feedback will be provided after submission to support improvement for later assessments.

Purpose

This assessment is the second stage of a larger project that continues from Assessment 1. In Assessment 1, the focus was on frontend design and usability for a Wordle-style and Word Search builder for Speech Pathology students and teachers. Assessment 2 extends that work by introducing the backend and database layer so that the application can store, manage, and retrieve word lists and activity settings more reliably.

The purpose of this stage is to move from a frontend-only builder to a data-driven application. Teachers should be able to create and manage phoneme-based word lists, save activity settings, and generate downloadable HTML outputs from stored data. This assessment prepares you to apply backend development, database design, API construction, and data persistence in the context of a specialist educational web application.

Task details

Design and implement the backend and database functionality for the Wordle and Word Search builder developed in Assessment 1. The application should allow teachers to enter, save, update, retrieve, and delete phoneme-based word content and related activity settings using a database-driven workflow.

Assessment 2 should support the generation of HTML activity outputs based on stored data rather than only temporary frontend values. The backend should provide the data needed to build Wordle and Word Search activities, including word lists, phoneme symbols, difficulty settings, hints, and any other metadata required by the application.

At this stage, the system should remain focused on Speech Pathology students and teachers. The builder should continue to generate activities for classroom use, but now the teacher's content should be stored and managed through the backend rather than being hard-coded into the frontend. The completed application should also be able to run inside a Docker container.

Instructions

You are required to:

- Develop the project from a Next.js app created with `npx create-next-app`.**
 - The project must be created using the Next.js starter workflow.
 - The application should then be extended with the required backend, database, and API functionality.
- Develop a backend application** that supports the frontend from Assessment 1.
 - Build server-side logic to handle activity data.
 - Ensure the frontend can communicate with the backend through APIs or routes.
 - Keep the backend aligned with the Wordle and Word Search builder use case.
- Dockerize the application** so that it runs in a Docker container and can be reproduced consistently in different environments. Use a Dockerfile structure similar to the one demonstrated in the relevant lab.

4. **Design and implement a database schema** that supports the project.
 - The database must allow the user to add a list of words and the phonemes for each word.
 - Phonemes may take up more than one character space, so the schema should store them appropriately.
 - The schema should represent phoneme-based word lists, activity type, difficulty level, hints, generated output settings, and any related metadata.
 - The schema should support storing and retrieving multiple activity configurations.
 - Use a suitable database tool or ORM such as Prisma.
5. **Provide CRUD functionality** for the data model.
 - Teachers should be able to create new word lists or activity sets.
 - Teachers should be able to read, update, and delete stored data.
 - The database should support the management of multiple Wordle or Word Search configurations.
 - CRUD operations for words in the database must be demonstrated in the video.
6. **Support the activity generation workflow.**
 - The backend should provide the data needed to generate downloadable HTML activity files.
 - The app should support both Wordle and Word Search outputs using stored data.
 - Word lists should be able to drive the generated content rather than being limited to a single fixed example.
7. **Implement validation and error handling.**
 - Validate user input before storing it in the database.
 - Handle missing, invalid, or malformed phoneme data gracefully.
 - Provide clear error messages where appropriate.
8. **Demonstrate the system in a video walkthrough.**
 - Show your student ID within the first 30 seconds.
 - Show your face and provide narration throughout the video.
 - Explain how the backend works and how the database supports the builder.
 - Demonstrate creating, saving, editing, reading, and deleting words or activity settings.
 - Show how the frontend uses backend data to generate Wordle and Word Search outputs.
 - Demonstrate the health API returning **200 OK** at `/health`.
 - Demonstrate the application running inside a Docker container.
9. **Produce a working technical submission.**
 - Submit the project code as a zip file.
 - Include your GitHub repository link.
 - Remove `node_modules` before uploading.
10. **Technical expectations:**
 - Use React, Next.js, and server-side development practices appropriately.
 - Apply modular and reusable code where possible.
 - Ensure code readability, maintainability, and reliability.
 - Demonstrate a backend and database architecture that can support later testing and deployment assessments.

Project continuity

This assessment builds directly on Assessment 1. The frontend builder and user interface should remain in place, but Assessment 2 adds the backend and database layer that make the builder more practical and reusable.

Assessment 1 established the interface for creating Wordle and Word Search activities. Assessment 2 now makes it possible to store phoneme-based word lists, manage activity settings, and generate downloadable HTML outputs from saved data. Later assessments will extend the project further by improving reliability, testing, and any additional deployment or presentation requirements.

Resources and readings relevant to the assessment

The following resources are especially relevant to this assessment:

- Next.js documentation
- React documentation
- Database and ORM documentation
- API and server-side development resources
- Lecture and tutorial materials on backend development, persistence, validation, and data handling

Assessment criteria

The rubric for this assessment will focus on backend functionality, database schema design, CRUD operations, validation, error handling, Docker execution, frontend-backend integration, code quality, and the clarity of the demonstration. The detailed rubric will be developed after the assessment brief is finalised.

Submission details

Submit:

- a zip file containing your project code and GitHub repository link
- any supporting documentation required by the unit coordinator

Remove `node_modules` before uploading if required by your submission process.

AI acknowledgement

If AI use is acceptable in this assessment, you must complete and submit an AI acknowledgement with your work. Failure to submit an acknowledgement when required may constitute a breach of academic integrity. You will find the AI acknowledgement on the Assessments page.

References

- Minimum five academic or industry sources
- APA 7th ed. style

Last modified: Friday, 17 July 2026, 2:16 PM

[← Assessment 1: 3-day short extension](#)

Jump to activity



[Assessment 2: Marking criteria and rubric →](#)

You are logged in as [Isaac Riley Lambert](#) ([Log out](#))

[Emergency information](#) [Child Safety](#) [Contacts](#) [Site map](#) [Accessibility](#) [Privacy](#) [Copyright and disclaimer](#)

© Copyright 2026

La Trobe University. All rights reserved. [La Trobe University CRICOS Provider Code Number 00115M](#)